

Cloud Infrastructure for AI

AICB-P2T2 · Ngày 16 · Chương 4: Hạ Tầng

Giảng viên Nguyễn Hải Dương · VinUniversity · Phase 2 · Track 2 · Tuần 4

"Ai đã từng deploy model lên cloud?

AWS? GCP? Azure? Local only?

Case study: *Startup burn \$50K/tháng GPU vì không optimize — right-size + spot instances, giảm còn \$12K.*

Cloud native hay on-premise — quyết định nào phù hợp với stack AI của bạn?"

Giữ câu hỏi này trong đầu suốt buổi học hôm nay

Nội Dung

1

So sánh Cloud Providers cho AI

2

Cloud Foundation (IaaS/PaaS/AI-aaS)

3

GPU Instance Types & Chi phí

4

Terraform IaC cho AI Stack

5

Docker → Kubernetes cho AI

6

Networking & Storage Strategy

7

Agent Infrastructure 8 Layers

8

AI Serving Stack (vLLM/SGLang)

Mục Tiêu Buổi Học

Sau buổi học này, bạn sẽ có thể:

1

Lựa chọn cloud provider phù hợp với AI workload cụ thể

2

Thiết kế GPU compute environment tối ưu chi phí

3

Triển khai container orchestration cho AI serving

4

Deploy AI endpoint production-ready trên cloud

Cloud providers → GPU analysis → Terraform IaC → K8s → AI Serving Stack → Demo

Deliverable Cuối Ngày

Cloud AI environment running + cost estimate + agent endpoint live

1

Cloud environment (AWS/GCP) đã setup với IAM least-privilege

2

GPU instance deployed trong private VPC qua Terraform

3

vLLM/SGLang endpoint chạy model inference thành công

4

Cost dashboard screenshot + cost estimate document

1

So Sánh Cloud Providers cho AI

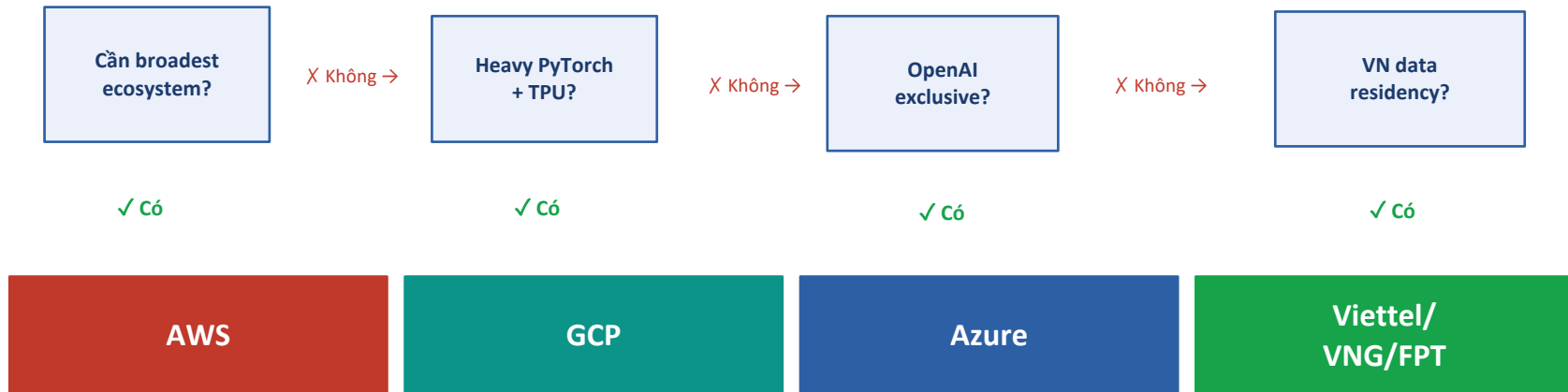
1. AWS vs GCP vs Azure: strengths, GPU flagships, và khi nào chọn
2. Vietnam cloud options: Viettel, VNG, FPT — data residency compliance
3. Specialized GPU clouds: Lambda, RunPod, CoreWeave — rẻ hơn 40-70%
4. Multi-cloud strategy & decision framework

Cloud Providers cho AI Workloads

Provider	GPU Flagship	Điểm Mạnh	Khi Nào Chọn
AWS	P5 (H100 8x), P5e (H200)	Ecosystem rộng nhất, Bedrock, SageMaker HyperPod	Broadest ecosystem + enterprise compliance
GCP	A3 Mega (H100 8x), TPU v5p	PyTorch/JAX, GKE GPU auto-provisioning, Vertex AI	Heavy PyTorch training + TPU interest
Azure	ND H100 v5, ND H200 v5	OpenAI Service exclusive, Prompt Flow LLMOps	Microsoft stack + OpenAI API
VN Cloud	T4/V100 (Viettel, VNG, FPT)	Giá 60–70% global, data residency ND13	Compliance ND13, data residency
Specialized	H100/H200 (Lambda, RunPod)	Rẻ hơn 40–70%, pure GPU, GMI Cloud \$2.10/hr H100	Cost-sensitive + team có infra skills

Chọn dựa trên: workload type × budget × compliance × latency requirements

Decision Framework: Chọn Cloud Nào?



Lưu ý: Nhiều tổ chức dùng **multi-cloud** — training ở provider A (GPU rẻ nhất), serving ở provider B (latency tốt nhất), data ở provider C (compliance). Cần abstraction layer (Terraform/Pulumi) để portable.

Vietnam Cloud & Specialized GPU Providers

Vietnam Cloud Options

Viettel Cloud: GPU T4/V100, giá ~60-70% global
VNG Cloud: đang build GPU capacity
FPT Cloud: data residency compliance
Ưu điểm: NĐ13/PDPD data residency
Limitation: chưa có H100/H200, limited AZ

Specialized GPU Clouds

Lambda Cloud: H100 @ \$2.49/hr
GMI Cloud: H100 @ \$2.10/hr (cheapest!)
RunPod: H100 @ \$3.35/hr
CoreWeave: dedicated GPU clusters
Rẻ hơn 40–70% vs hyperscalers



Trade-offs & Khi Nào Dùng

- Phù hợp: cost-sensitive teams, pure GPU workloads, team có infra skills
- Hạn chế: ít managed services, tự manage hơn, ít availability zones
- Rule: Specialized cloud cho training (cost), hyperscaler cho serving (managed scale + SLA)

2

Cloud Foundation cho AI

1. IaaS / PaaS / AI-aaS: phân biệt và khi nào dùng gì
2. Cloud-Native vs Cloud-Hosted: hybrid approach cho AI
3. Shared Responsibility Model trong AI context
4. Landing Zone: account structure, networking, guardrails

Mô Hình Cloud cho AI Workloads

AI-aaS	OpenAI API / Bedrock / Vertex AI (pay-per-token)	<i>Nhanh nhất, không cần infra</i>
PaaS	SageMaker / Azure ML / AI Platform (managed training + serving)	<i>Managed scaling, ít ops</i>
IaaS	EC2 GPU / GCE / Azure VM (full control, self-manage)	<i>Full GPU control, cần ops</i>
Physical	On-premise / Colocation (max control, max effort)	<i>Max control, max effort</i>

💡 AI thường hybrid: Training dùng IaaS (GPU control) → Serving dùng PaaS (managed scaling) → Prototype dùng AI-aaS (nhanh nhất)

Shared Responsibility & Landing Zone

Cloud Provider chịu trách nhiệm

Physical infrastructure + hypervisor
Network backbone & DDoS protection
Hardware maintenance & availability

Team bạn chịu trách nhiệm

Model security + data encryption
Access control (IAM least-privilege)
Prompt injection prevention
Data privacy compliance (NĐ13, GDPR)

Landing Zone — Setup đúng từ đầu (rework cost 10x!)

Account structure: workload accounts, shared services, security account
Networking: Transit Gateway hub-spoke topology, private subnets cho GPU
Centralized logging: CloudTrail + CloudWatch aggregation
Guardrails: SCPs (Service Control Policies) — enforce security baseline

3

GPU Instance Types & Chi Phí

1. GPU pricing 2026: T4, L40S, A100, H100, H200, B200
2. GPU Selection Decision Tree theo task + model size
3. MIG (Multi-Instance GPU) — maximize utilization
4. Cost strategy: Spot/Preemptible vs Reserved vs On-demand

GPU Pricing 2026 & So Sánh

GPU	VRAM	Giá/hr	Bandwidth	Use Case
T4	16 GB	\$0.35	320 GB/s	Inference nhỏ ($\leq 7B$)
L40S	48 GB	\$0.40–0.86	864 GB/s	Inference vừa — sleeper pick!
A10G	24 GB	\$1.00	600 GB/s	Inference production
A100	80 GB	\$1.79–2.70	2 TB/s	Fine-tuning (giảm từ \$3.0)
H100	80 GB HBM3	\$2.99–4.31	3.35 TB/s	Pre-training (giảm từ \$8.0!)
H200	141 GB HBM3e	\$3.72–5.58	4.8 TB/s	LLM 70B single GPU
B200	192 GB HBM3e	\$6.84–8.64	8 TB/s	Ultra-scale, limited avail.

Rule of thumb: inference $\rightarrow$ T4/L40S/A10G | fine-tuning $\rightarrow$ A100 | pre-training $\rightarrow$ H100 cluster

GPU Selection Decision Tree



Cost Strategy: Spot/Preemptible (tiết kiệm 60–70%) cho training jobs (interruptible). Reserved 1-year (giảm 40%) cho serving ổn định.

Ví dụ thực tế: GPT-2 1B token fine-tune → A100 on-demand=\$45 | Spot=\$14 | Reserved=\$27

GPU Instance Families — Dùng Dòng Nào?

Family	GPU	Khi Nào Dùng?	Tác Vụ AI
AWS P5 / P5e	H100 8x / H200	Multi-GPU training cluster	Pre-train, large fine-tune
AWS G5	A10G	Inference production	Serving $\leq 13B$ models
GCP A3 Mega	H100 8x	Distributed training	PyTorch DDP, DeepSpeed
GCP A3 Ultra	H200	Memory-intensive training	70B+ single-node
GCP TPU v5p	TPU v5p	JAX/XLA large-scale training	Massive-scale pre-train
Azure ND H100	H100	OpenAI fine-tuning	Azure ML pipelines
Azure ND H200	H200	High-memory inference	Large model serving

⚡ Karpenter (AWS): auto-provision đúng GPU type theo pod request | NAP (GCP): smarter Cluster Autoscaler | Scale-to-zero ngoài giờ: tiết kiệm 60%+ GPU idle cost

MIG, L40S & B200 — Advanced GPU Insights

MIG (Multi-Instance GPU)

A100/H100 chia tối đa 7 instances
1× A100 80GB → 7× 10GB instances
Isolated cho small model inference
Maximize utilization, serve nhiều models
Dùng khi: nhiều small models song song

L40S — Sleeper Pick

48GB VRAM, Ada Lovelace arch
FP8 Transformer Engine support
Giá: \$0.40–0.86/hr (3–5× rẻ hơn H100)
Throughput competitive cho ≤48GB models
Ideal: production inference small/medium

B200 Blackwell (2026)

192GB HBM3e, 8 TB/s bandwidth
Native FP4 support (mới)
11–15× inference vs H100 (promise)
Giá: \$6.84–8.64/hr (ramp-up 2026)
Early adopter premium, stabilize ~20-30% trên H200

Reserved vs Spot — Decision Framework

< 6 tháng → Spot/On-demand (linh hoạt, no commitment)
6–12 tháng → 1-year Reserved: tiết kiệm 30–40%
> 12 tháng → 3-year Reserved: tiết kiệm 50–60%
Training (interruptible) → Spot | Serving (stability needed) → Reserved | ROI dương sau ~6 tháng

4

Terraform Infrastructure-as-Code AI

1. Terraform modules cho GPU instances, VPC, security groups
2. State management: S3 backend + DynamoDB lock
3. Workspaces: dev / staging / prod isolation
4. Alternatives: Pulumi (Python/TS), OpenTofu, AWS CDK

Terraform cho AI Stack

```
resource "aws_instance" "gpu" {  
  instance_type = "g5.xlarge"  
  ami = "ami-nvidia-cuda-12"  
  
  root_block_device {  
    volume_size = 200  
    volume_type = "gp3"  
  }  
  
  tags = { Name = "ai-inference" }  
}
```

Infrastructure Setup

VPC: private subnet cho GPU, public cho LB
Security groups: 8080 (API), 6443 (K8s), 22 (SSH jump)
S3 backend + DynamoDB lock (team env)
Workspaces: dev / staging / prod

Alternatives to Terraform

Pulumi: Python/TypeScript native — popular với AI teams
OpenTofu: open-source fork (post HashiCorp BSL 2023)
AWS CDK: nếu pure AWS + TypeScript team
Tip: Dùng modules để reuse, workspaces để isolate

5

Container Orchestration: Docker → Kubernetes

1. Docker image optimization: multi-stage build 18GB → 6–8GB
2. NVIDIA GPU Operator: tự động install drivers + toolkit
3. Karenter (AWS) / NAP (GCP): smart GPU node provisioning
4. Init containers, namespaces, resource limits cho ML teams

Kubernetes Architecture cho AI Serving



Docker & K8s Best Practices cho AI

Docker Image Optimization

Base: nvcr.io/nvidia/cuda:12.1-runtime
Multi-stage build: 18GB → 6–8GB
Cache pip layer riêng (trước COPY source)
.dockerignore: exclude datasets, checkpoints
Result: cold start time giảm đáng kể

Kubernetes GPU Config

nvidia.com/gpu: 1 — requests = limits (always!)
GPU Operator: auto install drivers, toolkit, plugin
Init container: pre-download weights từ S3
Karpenter (AWS) / NAP (GCP): smart provisioning
Scale-to-zero ngoài giờ cao điểm

K8s Namespaces cho ML Teams — Best Practice

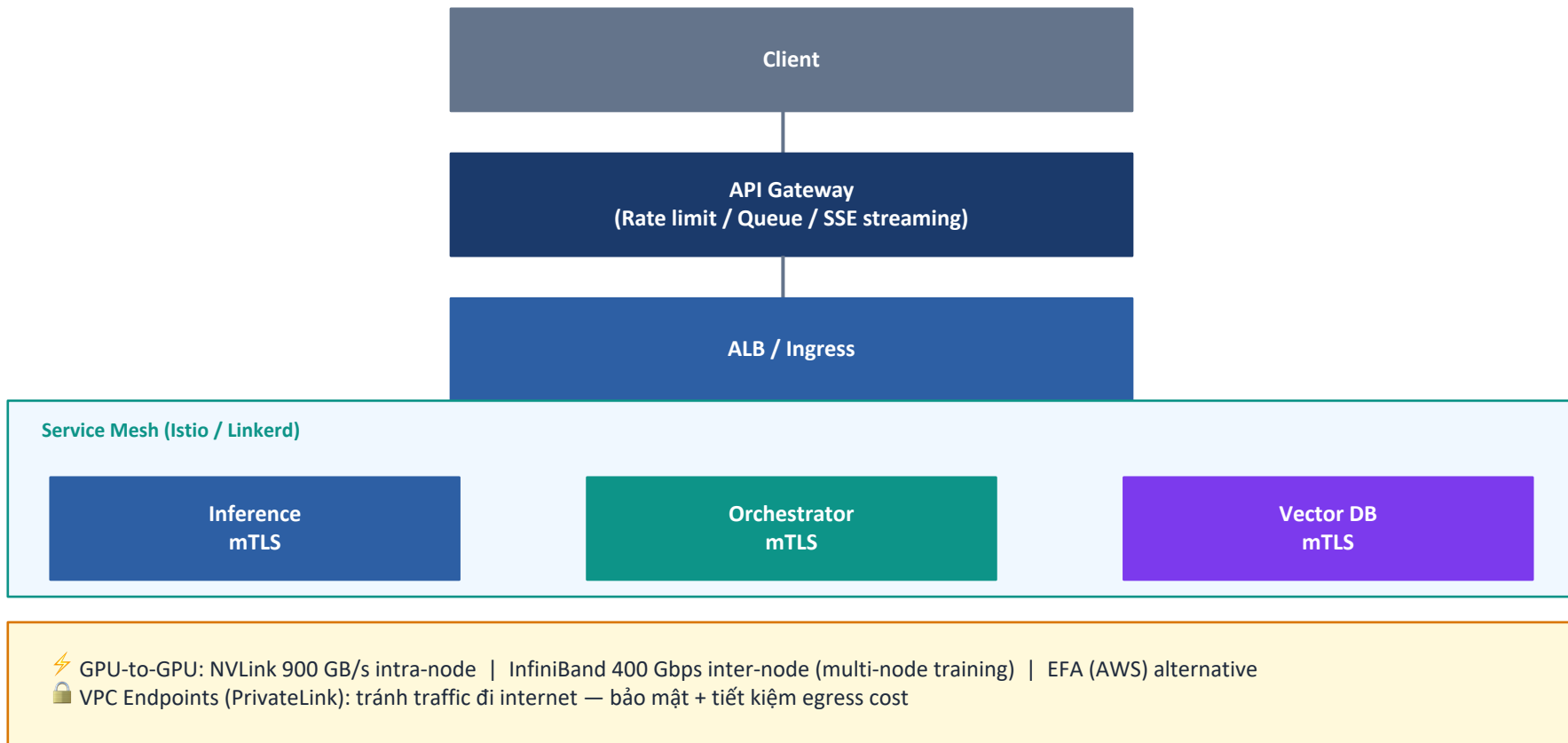
ml-training/ — separate resource quotas, cost tracking
ml-serving/ — separate RBAC, production isolation
ml-experiments/ — sandbox, no strict limits
⚠ NEVER overcommit GPU: fractional sharing phức tạp, dùng MIG thay vì overcommit

6

Networking & Storage Strategy

1. API Gateway patterns: rate limiting, streaming, timeout tuning
2. Service Mesh (Istio/Linkerd): mTLS, canary routing, tracing
3. GPU-to-GPU networking: NVLink 900 GB/s, InfiniBand 400 Gbps
4. Storage tiering: Hot (Redis) → Warm (S3) → Cold → Archive

Networking cho AI Workloads



Storage Strategy cho AI Systems

Hot	Redis / GPU Memory sub-ms latency	Active KV cache, embedding cache, session state
Warm	S3 Standard / EBS \$0.023/GB/mo	Model weights, recent checkpoints, training data
Cool	S3 Infrequent Access \$0.0125/GB/mo	Old checkpoints, infrequent datasets
Archive	S3 Glacier Deep Archive \$0.00099/GB	Compliance data, model archaeology



Storage Best Practices

S3 versioning cho model artifacts | Lifecycle policies: auto-archive sau 90 ngày | S3 Intelligent-Tiering cho mixed patterns

7

Agent Infrastructure: 8 Production Layers

1. Compute: GPU cho LLM inference, CPU cho orchestration, Serverless cho tools
2. Message Queue: Redis Streams vs Kafka vs RabbitMQ
3. Cache: L1 in-process LRU → L2 Redis → L3 Embedding cache
4. Observability: OpenTelemetry, LangSmith, Prometheus KPIs

8 Layers của Production AI Agent

8	Secrets & Config	Vault / AWS Secrets Manager, Feature flags cho A/B testing
7	Observability	OpenTelemetry → Jaeger traces, LangSmith, Prometheus KPIs
6	Networking	API Gateway, gRPC internal (high perf), HTTP+SSE (MCP transport)
5	Storage	PostgreSQL (conv history), pgvector (long-term), Redis (short-term TTL), S3 (tool outputs)
4	Cache	L1 in-process LRU dict L2 Redis shared (TTL) L3 Embedding cache
3	Message Queue	Redis Streams (low latency) Kafka (high throughput, replay) RabbitMQ
2	Orchestration	LangGraph / CrewAI / AutoGen on CPU pods — manages lifecycle & retry
1	Compute	GPU (LLM inference agents) + CPU (orchestrator) + Serverless (tool-calling)

Design principle: Stateless agents (externalize state → Redis/Postgres) cho horizontal scaling

Agent Infra: Chi Tiết Chọn Công Nghệ

Compute Pattern

GPU chỉ cho agents chạy LLM — orchestrator không cần GPU
Pattern: supervisor agent (CPU) dispatches → specialist agent (GPU)
Serverless (Lambda) cho lightweight tool-calling agents

Message Queue So Sánh

Redis Streams: low latency (<1ms), simple setup — best for most cases
Kafka: high throughput, durability, replay — large-scale agents
RabbitMQ: complex routing rules, dead letter queues

Cache — Multi-Level

L1: in-process LRU dict (fastest, per-instance)
L2: Redis shared across agents (TTL-based)
L3: Embedding cache (avoid re-embed same queries)
Target: 60–80% cache hit rate → giảm LLM API calls

Observability Stack

OpenTelemetry → Jaeger: distributed traces across agents
LangSmith / Weave: LLM-specific tracing & eval
Prometheus KPIs: tasks/min, error rate, avg latency, cost/request
HashiCorp Vault / Secrets Manager: API keys, credentials

8

AI Serving Stack: vLLM, SGLang & More

1. 6 serving engines 2026: vLLM, SGLang, LMDeploy, TensorRT-LLM, TGI, Ollama
2. SGLang RadixAttention: KV cache reuse — multi-turn gain 10–20%
3. LMDeploy TurboMind: 1.8× throughput vs vLLM (C++ zero overhead)
4. Deploy tips: GPU memory 80% safe zone, continuous batching always on

6 Serving Engines 2026 — So Sánh

Engine	Kỹ Thuật Chính	Ưu Điểm	Best For
vLLM	PagedAttention	Ecosystem rộng nhất, OpenAI-compatible API	Broad compatibility, easy deploy
SGLang	RadixAttention + Prefill-Decode disaggregation	Multi-turn +20%, JSON 3× faster, 400K+ GPUs globally	Agents + multi-turn chat + structured output
LMDeploy	TurboMind engine (C++) — zero Python overhead	1.8× throughput vs vLLM, Int4 2.4× faster	Quantized models, latency-sensitive apps
TensorRT-LLM	NVIDIA optimized kernels	30–50% faster cho high concurrency	Ultra-scale production (Perplexity-level)
TGI	HuggingFace native	Quick deploy, Prometheus built-in	Prototype nhanh, HF model ecosystem
Ollama	llama.cpp backend	CLI + local dev, easy model switching	Edge / laptop inference / development

2026 update: SGLang & LMDeploy đã vượt vLLM ~29% raw throughput cho nhiều use cases

SGLang & LMDeploy — Deep Dive 2026

◆ SGLang — RadixAttention

Powers 400,000+ GPUs globally (xAI Grok 3, Azure DeepSeek R1)
RadixAttention: reuse KV cache across requests có chung prefix
Multi-turn gain thêm 10–20% nhờ cache hits
Compressed FSM: JSON output 3× faster vs naive
Prefill-Decode Disaggregation: tách GPU roles
v0.4: zero-overhead batch scheduler (<2% CPU)

◆ LMDeploy — TurboMind Engine

TurboMind viết hoàn toàn bằng C++ — zero Python overhead
Persistent batch inference + blocked KV cache
1.8× request throughput vs vLLM baseline
Int4 inference: 2.4× faster than FP16
Best: quantized model deployment
Ideal: latency-sensitive production apps

⚙️ Practical Deploy Tips (áp dụng cho mọi engine)

GPU memory utilization 80% là safe zone — 95% gây CUDA OOM khi graph compilation
Continuous batching LUÔN bật — max_model_len tune theo actual usage (đừng set 128K nếu 99% requests <4K)
Health checks: /health endpoint, /v1/models verify model loaded, readiness probe 60s initial delay

Deploy vLLM / SGLang / TGI

vLLM — PagedAttention:

```
python -m vllm.entrypoints\
    .openai.api_server \
    --model meta-llama/llama-3-8B \
    --tensor-parallel-size 1 \
    --gpu-memory-utilization 0.80
```

SGLang — RadixAttention + JSON:

```
python -m sglang.launch_server \
    --model-path MODEL_PATH \
    --port 30000 \
    --tp 1
```



Practical Tips & Health Checks

GPU memory utilization 80% = safe zone (95% = CUDA OOM)

Continuous batching LUÔN bật | max_model_len tune theo actual usage

Health: GET /health | GET /v1/models | Readiness probe: initialDelaySeconds: 60

Compare TTFT: SGLang thường thấp hơn vLLM trong multi-turn nhờ RadixAttention cache hits

Tổng Kết — Key Takeaways

Những ý chính cần nhớ sau buổi học hôm nay

1

Cloud provider choice phụ thuộc workload type — không có "best", chỉ có "best fit". AWS (ecosystem), GCP (PyTorch/TPU), Azure (OpenAI), VN cloud (compliance).

2

H200 (141GB HBM3e) là new standard 2026. Terraform/Pulumi + Helm = reproducible infra. Tránh "works on my machine" syndrome — mọi thứ as code.

3

Serving stack 2026: vLLM, SGLang, LMDeploy, TensorRT-LLM, Ollama — chọn theo use case. SGLang cho agents/multi-turn, LMDeploy cho max throughput.

Tiếp Theo & Bài Tập

Ngày 17: Data Pipeline Engineering

"Airflow DAGs, Kafka streaming, ETL/ELT cho AI data — xây pipeline không để data bắn phá model"



Bài Tập & Chuẩn Bị

- ✓ Hoàn thành Lab 16: Cloud AI Environment Setup
- ✓ Cài đặt Docker Compose cho Airflow (pre-lab N17)
- 📖 Đọc trước: Apache Airflow TaskFlow API docs
- 🔗 lms.vinuni.edu.vn → Slide & templates trên LMS



Agenda gợi ý N17: Airflow fundamentals (60') → Kafka streaming (45') → ETL/ELT patterns (45') → Lab



Hỏi & Đáp

Câu hỏi nào về cloud providers, GPU selection, Kubernetes, hay AI serving stack?

Cảm ơn!

AICB-P2T2 · Ngày 16 · Cloud Infrastructure for AI
